

Experiment No.	6
Experiment Title	Breast Cancer Classification using Ensemble Methods (Bagging, AdaBoost, Gradient Boosting, and Stacking) ¹
Student Name	Preethi D
Register Number	3122247001046
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	20.08.2026

1 Objective

To classify breast tumours as malignant or benign using four ensemble learning strategies – Bagging, AdaBoost, Gradient Boosting, and Stacking – and to compare their performance after hyperparameter tuning using accuracy, precision, recall, F1-score, and ROC-AUC.

2 Problem Statement

Given the Wisconsin Diagnostic Breast Cancer dataset, where each sample is described by 30 numeric features computed from digitized images of a breast mass (radius, texture, perimeter, area, smoothness, etc.), the objective is to predict whether the tumour is malignant (M) or benign (B). Input: 30 numeric features per sample. Output: binary class label. Four ensemble strategies are trained, tuned via GridSearchCV, and compared: Bagging (with Decision Tree base estimators), AdaBoost (boosted decision stumps), Gradient Boosting, and a Stacking ensemble combining SVM, Naive Bayes, and Decision Tree base learners with a Logistic Regression meta-learner.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (wdbc.data)
Dataset Source	UCI Machine Learning Repository
Number of Samples	569
Number of Features	30 numeric features (mean, standard-error, and worst-case measurements of cell-nuclei characteristics), after dropping the id column
Number of Classes	2 (diagnosis: M = Malignant, encoded 0; B = Benign, encoded 1)
Missing Values	0
Train-Test Split	80% train / 20% test (stratified), giving 455 training and 114 test samples

4 Exploratory Data Analysis

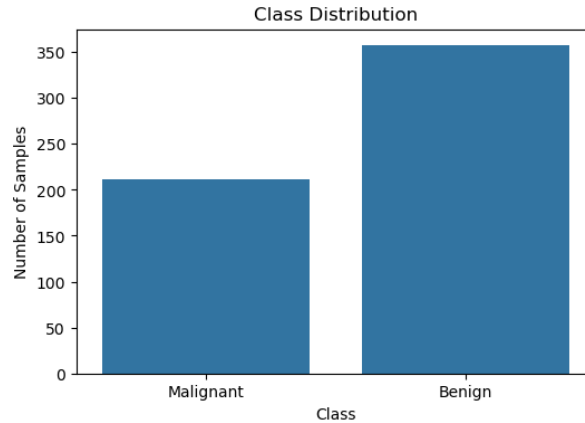


Figure 1: Class distribution of the Breast Cancer dataset

Inference: The dataset is moderately imbalanced, with 357 benign samples versus 212 malignant samples (roughly a 5:3 ratio). This is mild enough that accuracy remains informative, but precision, recall, F1-score, and ROC-AUC were all tracked to make sure the ensembles do not simply favour the majority (benign) class, since missing a malignant case is the costlier error clinically.

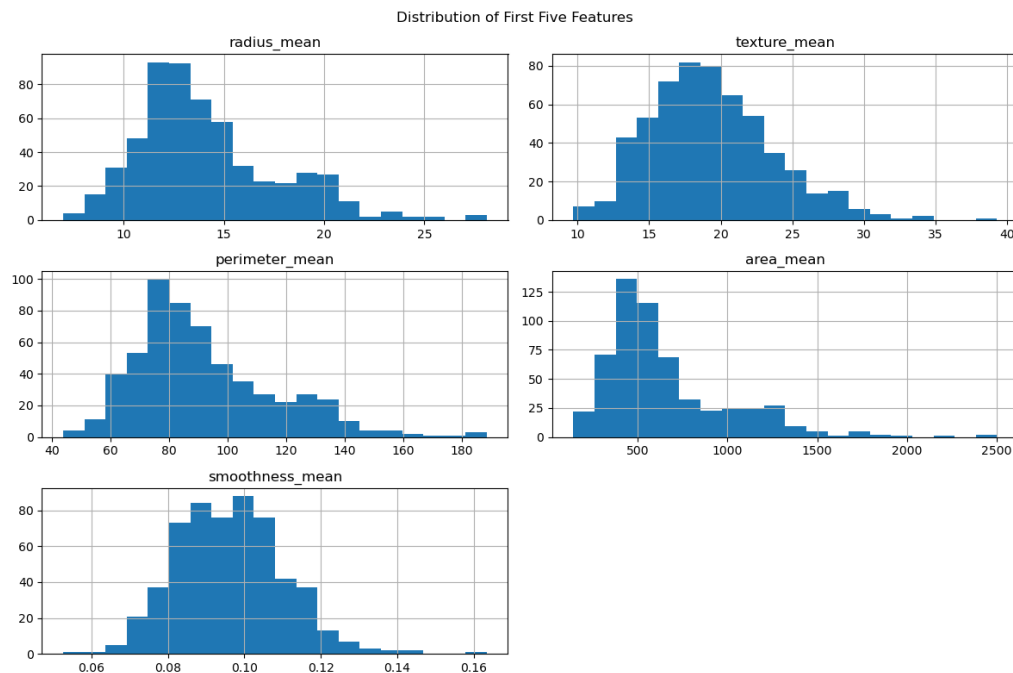


Figure 2: Distribution of the first five features (radius_mean, texture_mean, perimeter_mean, area_mean, smoothness_mean)

Inference: radius_mean, perimeter_mean, and area_mean are all right-skewed with a long tail of larger, more aggressive tumours, and are visibly related to one another since perimeter and area are geometric functions of radius. texture_mean and smoothness_mean are closer to symmetric, bell-shaped distributions. This mix of skewed and near-normal

features, along with the strong geometric relationships among some of them, motivated standardizing all features before training the SVM and Naive Bayes base learners used later in the Stacking ensemble.

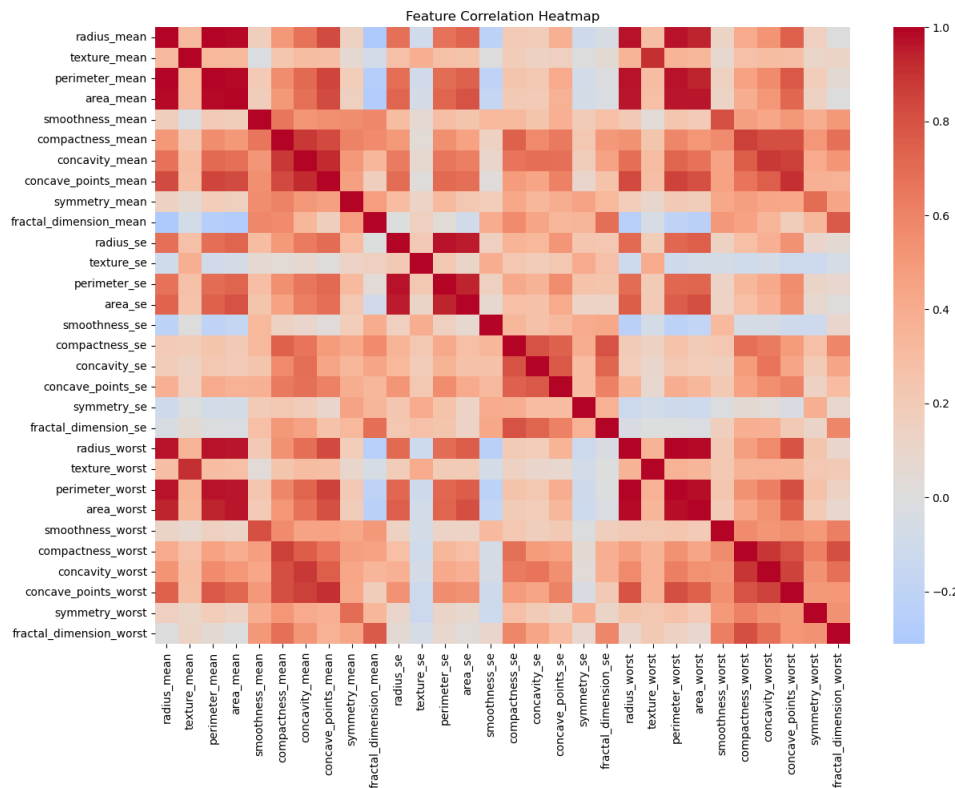


Figure 3: Correlation heatmap of all 30 features

Inference: Many features are extremely strongly correlated, especially the mean/standard-error/worst versions of the same underlying measurement (radius, perimeter, area, and the concavity-related features), since these are mathematically related quantities derived from the same cell-nuclei outlines. This heavy multicollinearity is well tolerated by the tree-based ensemble methods (Bagging, AdaBoost, Gradient Boosting) used here, though it is worth keeping in mind when interpreting the Stacking ensemble’s Logistic Regression meta-learner, which is more sensitive to correlated inputs.

5 Data Preprocessing

- **Missing value handling:** None required – the dataset has zero missing values across all 30 features.
- **Encoding:** The target column `diagnosis` was label-mapped from {M, B} to {0, 1} (M = Malignant = 0, B = Benign = 1); the non-informative `id` column was dropped from the feature set.
- **Feature scaling:** `StandardScaler` was fit on the training set and applied to both train and test sets for general use, and additionally embedded directly inside `Pipeline` objects for the SVM and Naive Bayes base learners used in the Stacking

ensemble, ensuring scaling is applied consistently even during cross-validation re-fitting. The tree-based ensembles (Bagging, AdaBoost, Gradient Boosting) do not require scaling and were trained on the unscaled features.

- **Train-test split:** An 80/20 stratified split was used (`train_test_split` with `stratify=y`, `random_state=42`), giving 455 training samples and 114 test samples.
- **Feature engineering:** None – all 30 original features were used directly as model inputs.

6 Machine Learning Algorithm

Bagging (Bootstrap Aggregating):

- **Working principle:** Bagging trains multiple copies of a base estimator (here, Decision Trees) on different bootstrap-resampled subsets of the training data and averages/votes their predictions, reducing variance without increasing bias.
- **Mathematical formulation:** $\hat{y} = \text{mode}\{h_1(x), \dots, h_T(x)\}$, where each h_t is trained on a bootstrap sample D_t drawn with replacement, and `max_samples`/`max_features` control the fraction of samples/features used per base estimator.
- **Advantages:** Reduces overfitting compared to a single Decision Tree, is easy to parallelize since base estimators are trained independently.
- **Limitations:** Because all base estimators are the same weak learner type trained somewhat similarly, gains plateau once enough estimators are used; it does not reduce bias, only variance.

AdaBoost (Adaptive Boosting):

- **Working principle:** AdaBoost trains a sequence of weak learners (here, depth-1 decision stumps), where each new learner focuses more on the samples misclassified by previous learners, and combines them into a weighted vote.
- **Mathematical formulation:** Final prediction $H(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$, where $\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$ and ϵ_t is the weighted error of weak learner h_t ; sample weights are increased for misclassified points after each round.
- **Advantages:** Can turn many weak learners into a strong classifier, often achieves high accuracy with simple base learners, and has few hyperparameters to tune.
- **Limitations:** Sensitive to noisy data and outliers, since misclassified (possibly noisy) points receive increasing weight each round, which can lead to overfitting if too many boosting rounds are used.

Gradient Boosting:

- **Working principle:** Gradient Boosting builds an additive ensemble of weak learners sequentially, where each new tree is fit to the negative gradient (residual errors) of the loss function with respect to the current ensemble's predictions.

- **Mathematical formulation:** $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$, where h_m is fit to approximate $-\left[\frac{\partial L(y, F(x))}{\partial F(x)}\right]_{F=F_{m-1}}$ and η is the learning rate.
- **Advantages:** Typically achieves very strong predictive performance, flexible with respect to loss function, and the learning rate/tree depth give fine-grained control over the bias-variance trade-off.
- **Limitations:** Sequential training makes it slower and harder to parallelize than Bagging, more hyperparameters to tune (learning rate, number of estimators, tree depth), and it is more prone to overfitting than Bagging if not carefully regularized.

Stacking:

- **Working principle:** Stacking trains several diverse base models (here, an RBF-kernel SVM, Gaussian Naive Bayes, and a depth-5 Decision Tree) and then trains a meta-learner (Logistic Regression) on their out-of-fold predictions to learn how to best combine them.
- **Mathematical formulation:** $\hat{y} = g(h_1(x), h_2(x), \dots, h_K(x))$, where $h_1, \dots, h_K$ are the base learners and g (the meta-learner) is trained on their cross-validated predictions to minimize the meta-level loss.
- **Advantages:** Can combine the complementary strengths of structurally different model types (a margin-based, a probabilistic, and a tree-based learner here), often outperforming any single base model.
- **Limitations:** More complex and computationally expensive (requires internal cross-validation to generate out-of-fold predictions for the meta-learner), harder to interpret than any individual base model, and results depend heavily on the choice and diversity of base learners.

7 Experimental Setup

Python Version	
Scikit-Learn Version	
IDE	Jupyter Notebook
Operating System	Windows
Hardware	

8 Hyperparameter Selection

GridSearchCV – Bagging (5-fold, scoring=accuracy):

Parameter	Value / Search Space
n_estimators searched	3, 5, 10, 20
max_samples searched	0.5, 0.8, 1.0
max_features searched	0.5, 0.8, 1.0
Best n_estimators	20
Best max_samples	0.5
Best max_features	0.5
Best CV accuracy	0.9582

GridSearchCV – AdaBoost (5-fold, scoring=accuracy):

Parameter	Value / Set
n_estimators searched	50, 100, 150
learning_rate searched	0.01, 0.1, 0.5
base estimator	Decision st
Best n_estimators	100
Best learning_rate	0.5
Best CV accuracy	0.9780

GridSearchCV – Gradient Boosting (5-fold, scoring=accuracy):

Parameter	Value
n_estimators searched	50, 100, 150
learning_rate searched	0.01, 0.1, 0.5
max_depth searched	1, 3, 5, 7, 9
Best n_estimators	100
Best learning_rate	0.5
Best max_depth	1
Best CV accuracy	0.9780

Stacking Ensemble configuration:

Parameter	Value
Base learners	SVM (RBF kernel, scaled),
Meta-learner	Logistic Regression (max_it
Internal CV for meta-features	5-fold
Avg CV accuracy	0.9626
Avg CV F1-score	0.9703

9 Results

Test set (114 samples), tuned models:

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Bagging	92.98%	95.71%	93.06%	94.37%	0.9897
AdaBoost	95.61%	94.67%	98.61%	96.60%	0.9835
Gradient Boosting	96.49%	95.95%	98.61%	97.26%	0.9947
Stacked Ensemble	97.37%	97.26%	98.61%	97.93%	0.9937

Best model overall: Stacked Ensemble (Accuracy 97.37%, Precision 97.26%, Recall 98.61%, F1-score 97.93%, ROC-AUC 0.994).

10 Result Visualization

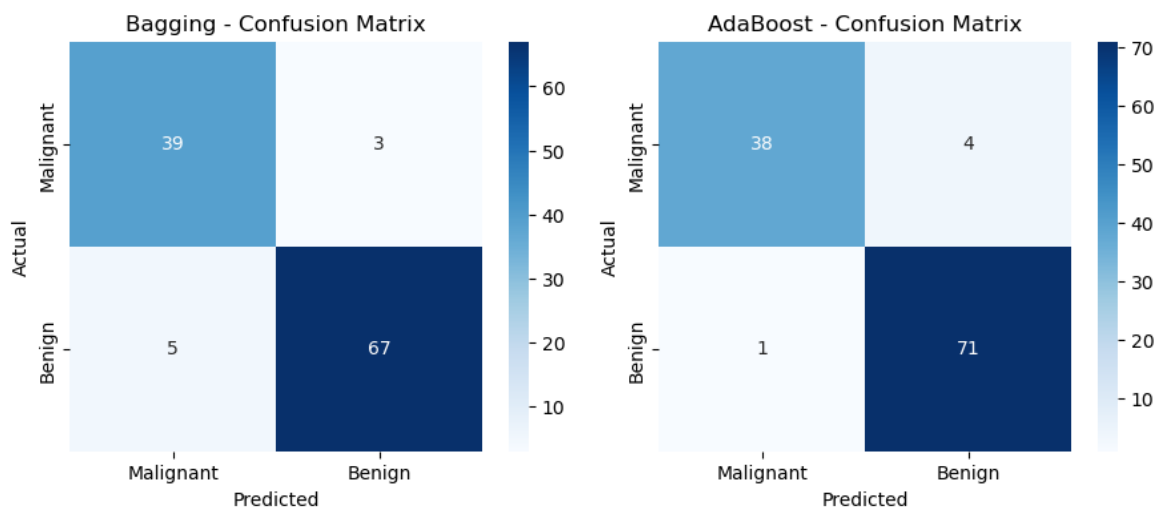


Figure 4: Confusion matrices: Bagging (left) and AdaBoost (right)

Inference: AdaBoost makes noticeably fewer false negatives than Bagging (1 vs. 5 malignant cases misclassified as benign), giving it a much higher recall (98.61% vs. 93.06%), though Bagging has slightly fewer false positives. For a diagnostic task, AdaBoost’s lower false-negative rate is clinically preferable even though its overall accuracy is only modestly higher.

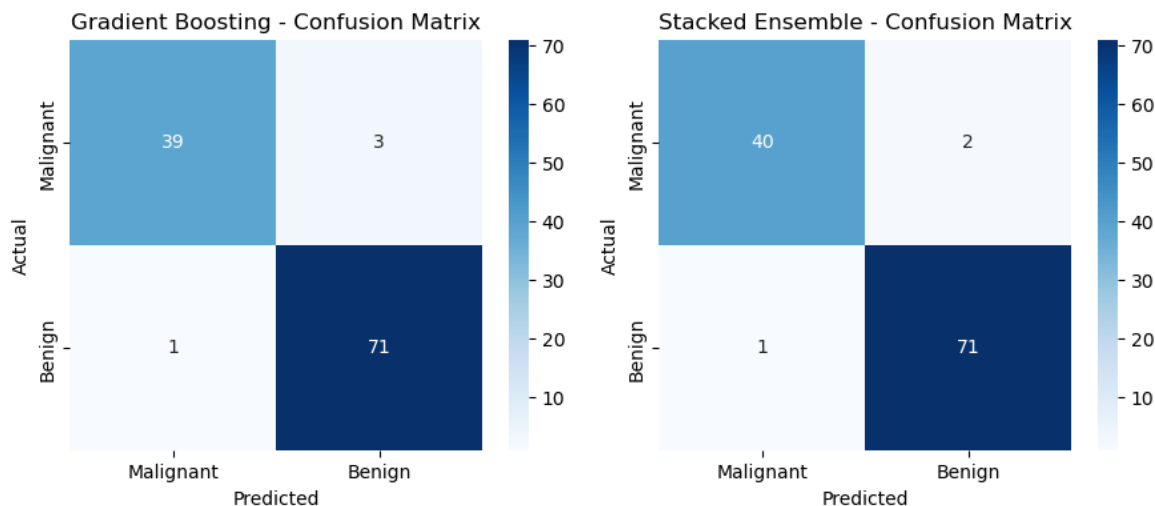


Figure 5: Confusion matrices: Gradient Boosting (left) and Stacked Ensemble (right)

Inference: The Stacked Ensemble achieves the fewest total misclassifications of all four models (3 errors out of 114), with a false-positive count lower than Gradient Boosting’s while matching its low false-negative count. This is consistent with Stacking’s ability to combine the complementary decision boundaries of SVM, Naive Bayes, and Decision Tree into a single, more accurate meta-prediction.

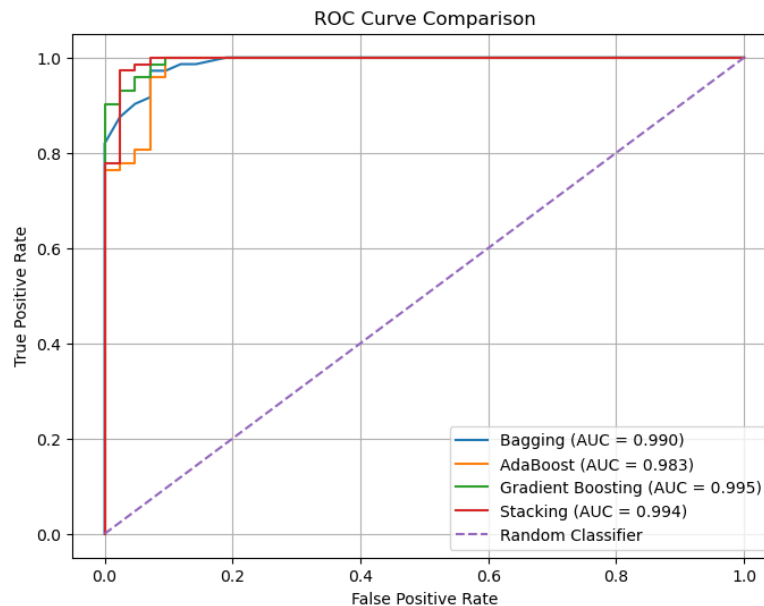


Figure 6: ROC curve comparison across all four ensemble models

Inference: All four ensembles achieve very high AUC values (0.98–0.99), with their curves closely hugging the top-left corner and clearly separated from the diagonal random-classifier line. Gradient Boosting attains the single highest AUC (0.9947), marginally ahead of Stacking (0.9937), indicating both models rank malignant cases above benign ones with near-perfect reliability across classification thresholds.

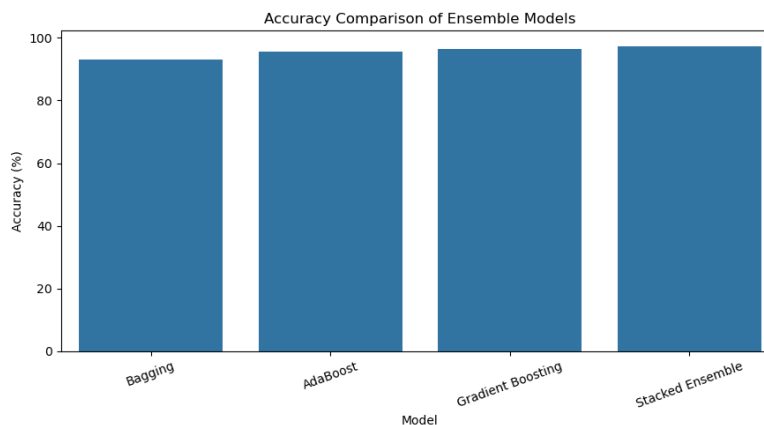


Figure 7: Accuracy comparison across ensemble models

Inference: Accuracy increases steadily from Bagging through AdaBoost and Gradient Boosting to Stacking, which achieves the highest bar at 97.37%. This ordering suggests that, on this dataset, boosting-style sequential error-correction (AdaBoost, Gradient Boosting) outperforms simple variance-reduction via Bagging, and that combining diverse model types via Stacking provides a further, smaller improvement on top of the best single ensemble.

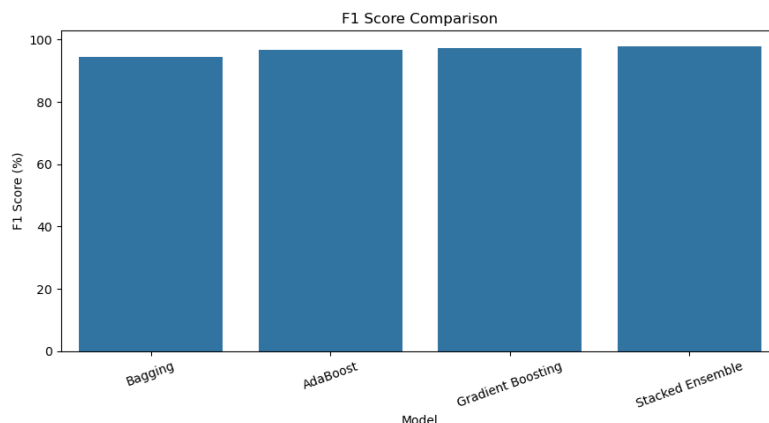


Figure 8: F1-score comparison across ensemble models

Inference: The F1-score comparison mirrors the accuracy comparison almost exactly, with the Stacked Ensemble again on top (97.93%), confirming that its improvement is not just in raw accuracy but reflects a genuinely better balance of precision and recall for the malignant/benign classification task.

11 Discussion

All four ensemble methods substantially outperformed what a single weak learner would typically achieve, with accuracies ranging from 92.98% (Bagging) to 97.37% (Stacking). Among the individual tree-based ensembles, boosting methods (AdaBoost, Gradient Boosting) outperformed Bagging, consistent with boosting’s ability to directly target and correct previous errors rather than simply averaging independently-trained trees; Gradient Boosting in particular achieved the best ROC-AUC (0.9947) of any single model. The Stacked Ensemble, combining SVM, Naive Bayes, and Decision Tree base learners under a Logistic Regression meta-learner, achieved the best overall accuracy, precision, and F1-score, demonstrating the value of combining structurally diverse models rather than relying on many variations of the same base learner (as in Bagging/Boosting). A limitation of this experiment is that the Stacking ensemble’s base learners were not individually tuned via grid search before being combined – tuning each base learner, or trying alternative meta-learners, could potentially close the small remaining gap to a perfect classifier further, though with diminishing returns and increased risk of overfitting on a dataset of this size (569 samples).

12 Conclusion

This experiment implemented and compared four ensemble learning strategies – Bagging, AdaBoost, Gradient Boosting, and Stacking – for malignant/benign breast tumour classification on the Wisconsin Diagnostic Breast Cancer dataset. After hyperparameter tuning via GridSearchCV, the Stacked Ensemble (SVM + Naive Bayes + Decision Tree base learners with a Logistic Regression meta-learner) achieved the best overall test performance (97.37% accuracy, 97.93% F1-score, 0.994 ROC-AUC), narrowly ahead of Gradient Boosting, confirming that combining diverse model types can outperform any single ensemble strategy on this task.

13 References

1. W. H. Wolberg, W. N. Street, and O. L. Mangasarian, “Breast Cancer Wisconsin (Diagnostic) Data Set,” UCI Machine Learning Repository, 1995.
2. F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
3. L. Breiman, “Bagging Predictors,” *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.
4. Y. Freund and R. E. Schapire, “A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
5. J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
6. D. H. Wolpert, “Stacked Generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.